

DOKUMEN TOOLS, FRAMEWORK, LIBRARY, DAN SKALABILITAS SMARTSTOCK PRO

1. Pendahuluan

Dokumen ini menjelaskan tools, framework, library, dan pertimbangan skalabilitas yang digunakan pada SmartStock Pro. Tujuan dokumen ini adalah memberikan gambaran teknis kepada asesor atau pelanggan mengenai alasan pemilihan teknologi, kemampuan pengembangan, dan strategi peningkatan kapasitas sistem.

SmartStock Pro adalah MVP/demo sistem manajemen inventaris multi gudang berbasis web. Fitur inti seperti login, role access, CRUD master data, transaksi stok masuk/keluar, dashboard, audit log, upload gambar produk, export CSV, PDF via print browser, dan health check telah tersedia. Fitur advanced seperti import CSV/Excel, queue jobs, transfer stok antar gudang, notifikasi real-time, dan FIFO/LIFO batch layer masih dicatat sebagai simulasi MVP atau rancangan pengembangan lanjutan.

2. Ringkasan Teknologi

Komponen	Teknologi	Fungsi	Alasan Pemilihan
Frontend	React.js	Membangun UI interaktif berbasis komponen	Cocok untuk dashboard, form CRUD, dan SPA.
Build Tool	Vite	Development server dan bundling frontend	Cepat, ringan, dan mudah dikonfigurasi.
Backend	Laravel	REST API, MVC, validasi, middleware, auth	Framework stabil untuk aplikasi CRUD dan role access.

Database	PostgreSQL	Penyimpanan data relational	Cocok untuk relasi produk, gudang, supplier, transaksi, dan audit log.
Authentication	Laravel Sanctum	Token-based authentication	Cocok untuk SPA yang mengakses API dengan Bearer Token.
Chart	Recharts	Visualisasi grafik dashboard	Integrasi mudah dengan React.
HTTP Client	Axios	Request API dari frontend	Mendukung interceptor Bearer Token dan response handling.
Routing Frontend	React Router DOM	Routing halaman frontend	Mendukung protected route dan navigasi SPA.
Version Control	Git/GitHub	Kontrol versi dan kolaborasi	Memudahkan tracking perubahan, branch, dan rollback.
PHP Dependency	Composer	Mengelola package Laravel	Standar dependency manager PHP.
JS Dependency	NPM	Mengelola package frontend	Standar dependency

			manager JavaScript.
--	--	--	------------------------

3. Analisis Pemilihan Frontend

Frontend menggunakan React.js dan Vite karena beberapa alasan:

- **Single Page Application (SPA)**

Aplikasi dapat berpindah halaman tanpa reload penuh sehingga pengalaman pengguna lebih cepat.

- **Reusable Component**

Komponen seperti layout, sidebar, navbar, table, modal, alert, form produk, form gudang, dan form transaksi dapat digunakan berulang.

- **Cepat untuk Development**

Vite menyediakan development server yang cepat dan hot reload, sehingga perubahan UI dapat diuji dengan efisien.

- **Cocok untuk Dashboard**

React memudahkan pembuatan dashboard interaktif, chart, card ringkasan, dan tabel data.

- **Integrasi REST API**

React dapat berkomunikasi dengan Laravel REST API menggunakan Axios dan Bearer Token.

4. Analisis Pemilihan Backend

Backend menggunakan Laravel REST API karena:

- **Struktur MVC**

Laravel memiliki pola Model-View-Controller yang memisahkan data, logic, dan request handling.

- **Routing API Jelas**

Endpoint seperti `/api/products`, `/api/warehouses`, dan `/api/stock-transactions` mudah dikelola.

- **Middleware**

Middleware digunakan untuk autentikasi Sanctum dan pembatasan role.

- **Eloquent ORM**

Eloquent membantu mengakses database secara aman dan lebih terstruktur.

- **Validasi Request**

Laravel memiliki validasi bawaan untuk input produk, supplier, transaksi stok, upload gambar, dan login.

- **Laravel Sanctum**

Sanctum digunakan untuk token authentication pada SPA.

- **Cocok untuk CRUD dan Role Access**

Laravel memiliki ekosistem yang kuat untuk aplikasi bisnis berbasis CRUD.

5. Analisis Pemilihan Database

PostgreSQL dipilih karena:

- Merupakan relational database yang stabil.
- Cocok untuk data inventory yang memiliki banyak relasi.
- Mendukung integritas data melalui foreign key.
- Mendukung query agregasi untuk dashboard seperti total stok masuk, total stok keluar, dan chart per kategori.
- Dapat ditingkatkan performanya dengan indexing dan query optimization.

Data SmartStock Pro bersifat company-wide/global. Artinya master data seperti produk, kategori, gudang, dan supplier tidak diperlakukan sebagai data personal per user.

`user_id` digunakan sebagai penanda pembuat data atau pelaku aktivitas, bukan pembatas visibility data perusahaan.

6. Dokumentasi Library Pihak Ketiga

Library	Versi	Fungsi	Lisensi	Alasan Penggunaan
axios	Mengikuti package.json	HTTP client frontend	Umumnya MIT	Memudahkan request API dan Bearer Token.
react-router-dom	Mengikuti package.json	Routing frontend	Umumnya MIT	Navigasi SPA dan protected route.
recharts	Mengikuti package.json	Chart dashboard	Umumnya MIT	Visualisasi data stok berbasis React.
laravel/sanctum	Mengikuti composer.json	Authentication token	MIT	Token auth untuk SPA dan REST API.
Laravel Framework	Mengikuti composer.json	Backend framework	MIT	MVC, routing, middleware, ORM, validasi.
Vite	Mengikuti package.json	Build tool frontend	MIT	Development server cepat dan bundling.

PostgreSQL driver	Mengikuti konfigurasi PHP/Laravel	Koneksi database PostgreSQL	Tergantung driver	Menghubungkan Laravel ke PostgreSQL.
-------------------	-----------------------------------	-----------------------------	-------------------	--------------------------------------

7. Skalabilitas Sistem

Strategi skalabilitas SmartStock Pro:

- **Pemisahan Frontend dan Backend**

Frontend dan backend dipisahkan sehingga dapat dideploy pada server berbeda.

- **REST API Dapat Dipisah Server**

Laravel API dapat dipindahkan ke application server khusus ketika traffic meningkat.

- **Indexing Database**

Kolom yang sering digunakan untuk filter seperti `product_id`, `warehouse_id`, `type`, dan `transaction_date` dapat diberi index.

- **Pagination**

Untuk data besar, tabel produk, transaksi, supplier, dan audit log perlu menggunakan pagination.

- **Caching**

Data dashboard yang sering dibaca dapat dicache untuk mengurangi beban query.

- **Queue**

Import data dan generate laporan besar dapat dipindahkan ke queue. Saat ini queue jobs masih simulasi MVP.

- **File Storage Terpisah**

Jika gambar produk bertambah banyak, file dapat dipindahkan ke object storage/cloud storage.

- **Load Balancer**

Untuk skala besar, beberapa application server dapat ditempatkan di belakang load balancer.

8. Analisis Peningkatan Jumlah Pengguna

10 User Internal

Pada skenario 10 user internal, server minimal 2 core CPU dan 4 GB RAM masih cukup untuk MVP. Beban utama berasal dari login, CRUD, dan transaksi stok.

50 User Gudang

Pada 50 user gudang, jumlah request transaksi dan pembacaan dashboard meningkat. Database perlu index dasar pada tabel transaksi. Monitoring log mulai penting untuk melihat query lambat.

100+ User Perusahaan

Pada 100+ user, perlu strategi tambahan seperti:

- application server terpisah,
- database tuning,
- pagination semua tabel besar,
- caching dashboard,
- queue untuk laporan besar,
- file storage terpisah,
- monitoring response time.

9. Analisis Peningkatan Volume Data

Peningkatan volume data dapat terjadi pada:

- jumlah produk,
- transaksi stok,

- audit log,
- file gambar,
- laporan historis.

Strategi teknis:

- **Pagination** untuk daftar data besar.
- **Indexing** pada kolom filter dan foreign key.
- **Archive Audit Log** untuk data audit lama.
- **Background Export** untuk laporan besar.
- **Query Optimization** untuk dashboard dan laporan.

10. Maintainability

Maintainability dijaga melalui:

- **Struktur Folder Backend**

Controller, model, migration, seeder, middleware, dan route API dipisahkan sesuai tanggung jawab.

- **Struktur Folder Frontend**

Halaman, komponen, API services, context, dan utility role dipisahkan.

- **Reusable Components**

Komponen seperti DashboardLayout, Sidebar, Navbar, DataTable, Modal, Alert, dan form digunakan ulang.

- **API Services**

Frontend memiliki file API terpisah seperti `productApi.js`, `warehouseApi.js`, `supplierApi.js`, dan `transactionApi.js`.

- **Separation of Concerns**

UI, request API, role logic, dan backend business logic dipisahkan.

- **Git Version Control**

Git digunakan untuk tracking perubahan dan rollback.

11. Risiko Skalabilitas dan Mitigasi

Risiko	Dampak	Mitigasi
Transaksi stok terlalu banyak	Query lambat pada dashboard dan laporan	Tambahkan index, pagination, dan query optimization.
Laporan besar menyebabkan timeout	Export gagal atau UI menunggu terlalu lama	Gunakan background job dan queue pada pengembangan lanjutan.
File gambar produk banyak	Storage penuh dan backup membesar	Gunakan object storage/cloud storage dan kompresi gambar.
Audit log tumbuh cepat	Database membesar dan query log lambat	Archive audit log lama dan gunakan pagination.
Banyak user bersamaan	Response API lambat	Pisahkan server, gunakan caching, dan load balancer.
Database menjadi single point of failure	Sistem tidak bisa digunakan jika database down	Backup, replication, dan monitoring database.

12. Kesimpulan

SmartStock Pro menggunakan teknologi yang sesuai untuk MVP sistem inventaris berbasis web. React + Vite mempercepat pengembangan frontend, Laravel REST API

menyediakan struktur backend yang kuat, PostgreSQL mendukung data relational inventaris, dan Sanctum menyediakan autentikasi token. Sistem sudah siap untuk demo BNSP pada fitur inti, sementara beberapa aspek skalabilitas seperti pagination penuh, queue jobs, background export, dan storage terpisah menjadi rancangan pengembangan lanjutan.